

Wait / Block / Wakeup

条件不满足时，执行权怎样安全暂存、排队，并在事件后重新竞争 CPU

408 · OS-B01

Mother Interface

Condition Unsatisfied → Register Wait + Block Safely
→ Event / State Change → Wake Eligible Waiter
→ Ready/Runnable → Scheduler/Dispatch
→ Reacquire Protection + Recheck Condition

本 Bridge 专门拥有“同步条件、等待队列（Wait Queue）与任务状态（Task State）”之间的控制权交接契约；同步原语语义（OS-03）、设备中断处理（OS-05）和调度算法（OS-01/02）仍由各自专题独立拥有。

先定义接口两侧的词

- **Condition Predicate（条件谓词）**：决定当前执行流能否继续推进的条件，如 `count > 0`、“request completed”或“lock available”。谓词由拥有共享状态的 Topic 定义，不能用“收到了 wakeup”替代。
- **Wait Relation / Wait Queue（等待关系/等待集合）**：记录“哪些执行实体正在等待哪个条件/事件”。实现可以是链表、队列、哈希等待桶或其他结构；Bridge 只依赖等待者能够被事件定位这一语义。
- **Task State（任务执行状态）**：描述执行实体当前是否具备 CPU 调度资格。408 可使用 Ready/Running/Blocked；Linux 的 `TASK_*` 状态只是一个工程实例，不能反过来当跨系统定义。
- **Wakeup（唤醒）**：移除或更新等待关系，使满足资格的实体重新成为 Ready/Runnable。它改变的是调度资格，不是“立即获得 CPU”，也不保证条件到真正运行时仍然成立。

Owners 与职责边界

- **左侧 Owner（OS-03 并发同步/OS-05 I/O）**：拥有锁、条件变量、信号量或外设 DMA 完成事件本体。
- **右侧 Owner（OS-01/02 进程与调度）**：拥有 `task_struct`、就绪队列（Run Queue）、CPU 时间片与上下文切换机制。
- **本 Bridge Owns**：条件检查失败后“如何安全入队让出 CPU”、事件到达后“如何唤醒并重检条件”的协议链。
- **Stops**：不重新定义死锁避免算法、PV 信号量数值含义或外设寄存器细节。

1 核心机制：为什么必须“先入队、改状态，后释放保护”

在并发系统与操作系统内核中，条件检查与休眠之间存在极易触发的严重竞争态——丢失唤醒（Lost Wakeup）。

若线程 A 先检查条件为假，在准备将自己标为 Blocked 之前发生上下文切换；此时线程 B 生产了资源并调用 `wakeup()`。由于线程 A 尚未挂入等待队列，线程 B 认为“当前没有等待者”因而直接返回。随后线程 A 恢复执行，盲目把自己标为 Blocked 并让出 CPU，陷入永久无法被唤醒的死锁状态！

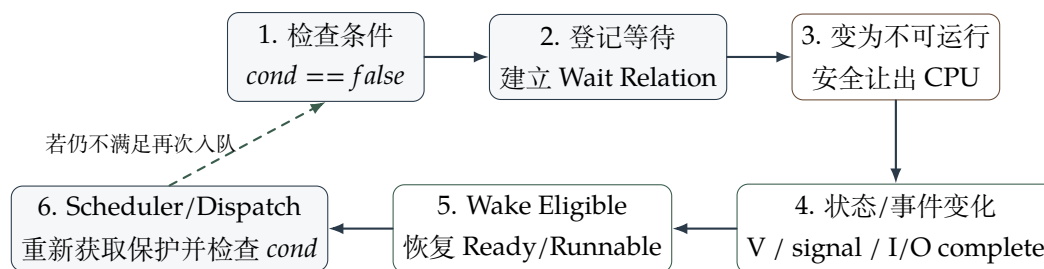
1.1 标准安全休眠与唤醒协议

安全等待协议真正要求的不是固定 API 顺序，而是一个原子性不变量：从“条件仍为假”到“等待者已经可被未来事件找到并且不会继续消费 CPU”之间，不能出现 `wakeup` 无人接收的窗口。一种典型实现会：

1. 在保护共享谓词的同步范围内检查条件；
2. 建立当前执行实体与等待条件之间的可定位关系；
3. 把执行实体变为不可运行/等待状态，并与释放保护、让出 CPU 做成不会丢失事件的协议；
4. 进入调度路径，让其他 Ready/Runnable 实体继续推进。

Linux wait queue、条件变量的 `atomic release-and-wait`、semaphore 的 P 操作都可以实现这份契约，但具体函数名、状态编码和内部队列结构不属于本 Bridge。

2 母模型：阻塞排队与事件唤醒流转拓扑



2.1 为什么条件等待通常要在返回后重新检查谓词

当线程从阻塞中被唤醒并真正重新获得 CPU 时，它面对的条件**不保证依然为真**：

- **多等待者竞争（典型 Mesa 风格语义）**：一个状态变化可能使若干等待者重新有资格竞争，但先运行者可能在后运行者取得锁之前再次改变共享状态；
- **允许虚假/非唯一原因唤醒的接口**：某些条件等待 API 明确允许调用者在没有“一次通知对应一次永久真条件”的情况下返回。

因此，对条件变量/谓词式等待，稳健模板是 `while(!condition) { wait(); }`；`wake` 只意味着“值得再检查”。若题目给的是计数信号量、一次性完成事件等更强接口，应按该 Owner 的契约判断，不把 `while` 机械推广到所有等待原语。

3 四种等待策略全景对比

等待策略	核心机制与状态	优点与适用场景	劣势与工程代价
自旋等待 (Spin)	执行实体仍可运行，在 CPU 上反复检查条件	避免睡眠/唤醒路径；当预计等待时间短于阻塞切换成本时可能划算	等待期间持续消耗 CPU；若持有者没有机会运行，继续自旋只会放大停滞
阻塞休眠 (Blocking)	建立等待关系并失去 CPU 调度资格，直到事件使其重新 Ready	CPU 可交给其他 runnable 工作；适合相对较长或由外部事件决定的等待	至少支付进入等待与以后重新运行的调度/现场管理成本，且有 wake-to-run 延迟
让出执行 (Yield)	当前实体主动放弃本次 CPU 机会，但通常没有声明自己在等待哪个确定事件	可降低原地独占，给其他 runnable 实体机会	仍可能很快再次被选中；不能替代真正的事件等待协议
挂起 (Suspend)	在教材模型中把“可运行/阻塞”再叠加一个暂不参与正常调度的维度	可表达外部暂停、负荷调节或经典换出场景	Suspend 与 Blocked 正交；现代系统不应机械理解为“整个进程映像必然完整写入 swap”

4 最小执行轨迹：生产者-消费者队列交接

设有限缓冲区当前为空 (count == 0)：

1. 消费者 C1 到达：获取互斥锁，检查 count == 0 为真；C1 将自身放入 not_empty 等待队列，释放互斥锁并转入 BLOCKED；
2. 生产者 P1 到达：获取互斥锁，向缓冲区写入一个元素，count = 1；
3. P1 触发唤醒：P1 在改变 count 后通知 not_empty 等待者，C1 的等待关系被解除并重新成为 Ready/Runnable；
4. 调度与重验：P1 释放互斥锁并退出。调度器选中 C1，C1 恢复执行，首先重新争抢互斥锁；获取锁后重新检查 count > 0 成立，成功取出元素并减少 count。

Anti-Bridge：三个必须阻断的错误直觉

- Wakeup ≠ Immediate Execution：唤醒只是赋予调度资格，不代表立刻夺取正在运行任务的 CPU。
- Device Interrupt ≠ Task Rescheduled：外设中断完成并唤醒等待线程后，CPU 仅将任务标记就绪，真正的上下文切换发生在该中断返回或下一个调度点。
- Wakeup ≠ Condition True：被唤醒不代表资源已属于你，必须在重新获取锁后再次验证条件。

30 秒极速调用协议

做题遇到“进程等待事件/信号量/I/O”时，严格按三步追踪：

谁在等待哪个条件？

Event 使其 Ready/Runnable

调度后按接口契约重新验证条件